

Probabilistic Machine Learning and Deep Learning

Gabriele Granzotto, Sara Trabucco, Davide Zorzetto

22 Luglio 2026

1 Masked Diffusion Language Model

The reference is the paper *Simple and Effective Masked Diffusion Language Models* [1], whose main claim is that **masked discrete diffusion can be reduced to a simple weighted cross-entropy objective**.

The main idea is:

- an **autoregressive language model** factorizes a sequence probability as:

$$p_{\theta}(\mathbf{x}) = p_{\theta}(x^{1:L}) = \prod_{i=1}^L p_{\theta}(x^i | x^{<i})$$

and therefore generates one token at a time from left to right;

- a **diffusion trajectory** constructs a sequence of increasingly corrupted versions of a clean sequence:

$$\mathbf{x} = \mathbf{z}_0 \rightarrow \mathbf{z}_t \rightarrow \mathbf{z}_1$$

where $\mathbf{z}_0 = \mathbf{x} = (x^1, \dots, x^L)$ is the clean sequence, $\mathbf{z}_t = (z_t^1, \dots, z_t^L)$ is its corrupted version at time t and $\mathbf{z}_1 = (m, \dots, m)$ is fully masked.

We slightly modify the paper's notation: $x^i \in \mathcal{V}$ as a clean token, where $|\mathcal{V}| = K$ is our *vocabulary size*, while $m \notin \mathcal{V}$ is the special token corresponding to **[MASK]**; this makes it so we are considering the extended vocabulary $\bar{\mathcal{V}} = \mathcal{V} \cup \{m\}$. $\mathcal{V}$ is the finite set of tokens and each token $v \in \bar{\mathcal{V}}$ can be represented by a one-hot vector $\mathbf{e}(v) \in \{0, 1\}^{K+1}$.

Under the SUBS parameterization (which will be discussed later) we have that $\Delta_{SUBS}^{K+1} = \{\bar{p} \in \Delta^{K+1} : \bar{p}_m = 0\}$. We have that Δ^K is isomorphic to Δ_{SUBS}^{K+1} , where the mask occupies the final coordinate.

What the model outputs is the *probability distribution over the vocabulary*, so $x_{\theta}^i(\mathbf{z}_t, t) \in \Delta^K \cong \Delta_{SUBS}^{K+1}$ (where Δ^K is the *probability simplex*, the set of probability vectors over K categories, not counting **[MASK]**).

For example, assume we have a clean vocabulary of size $K = 3, \mathcal{V} = \{\text{cat}, \text{dog}, \text{house}\}$ and **[MASK]**; we will have then:

$$\{\mathbf{e}(v) : v \in \bar{\mathcal{V}}\} = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\}$$

We will have then that an element of Δ^{K+1} is for example:

$$p = \begin{bmatrix} 0.6 \\ 0.2 \\ 0.2 \\ 0 \end{bmatrix} \in \Delta^4$$

which represents the probabilities $P(\text{cat}) = 0.6$, etc.

So the denoising model is defined as:

$$\mathbf{x}_{\theta} : \bar{\mathcal{V}}^L \times [0, 1] \rightarrow (\Delta^K)^L$$

which means that the model receives $(\mathbf{z}_t, t)$, which is the observed sequence of length L and the diffusion time t , and outputs $x_{\theta}^i(\mathbf{z}_t, t) \in \Delta^K$ for every position $i = 1, \dots, L$, which is a probability distribution over the vocabulary:

$$[x_{\theta}^i(\mathbf{z}_t, t)]_v = p_{\theta}(x^i = v | \mathbf{z}_t, t) \quad v \in \mathcal{V}$$

and, as we will discuss later, under SUBS $[x_\theta^i(\mathbf{z}_t, t)]_m = 0$.

1.1 Forward process

Having defined the state space and denoising model, the goal is now defining the **forward corruption process** that generated noisy training inputs.

We define $\alpha_t = P(z_t^i = x^i | x^i)$ the **noise schedule**, such that $\alpha_0 = 1$ and $\alpha_1 = 0$: at $t = 0$ the sequence is all clean ($\mathbf{z}_0 = \mathbf{x}$), increasing t increases the corruption and $t = 1$ means that the sequence is fully masked ($\mathbf{z}_1 = (m, \dots, m)$).

Why do we consider the marginal forward process?

During training, the diffusion time t is sampled directly. We therefore need to sample $\mathbf{z}_t$ from $\mathbf{x}$ without simulating all the intermediate forward transitions. Because of this, we need to consider the marginal distribution $q(\mathbf{z}_t | \mathbf{x})$; at token-level, this is defined as:

$$q(z_t^i | x^i) = \text{Cat}(z_t^i | \alpha_t \mathbf{e}(x^i) + (1 - \alpha_t) \mathbf{e}(m))$$

The product will tell us how to sample the noisy sequence **directly at time** t from the clean one.

In the previous example (with $|\bar{\mathcal{V}}| = K + 1 = 4$), we have for $x^i = \text{dog}$:

$$\mathbf{e}(x^i) = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \mathbf{e}(m) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

so

$$\alpha_t \mathbf{e}(x^i) + (1 - \alpha_t) \mathbf{e}(m) = \begin{bmatrix} 0 \\ \alpha_t \\ 0 \\ 1 - \alpha_t \end{bmatrix}$$

More generally, among the $K + 1$ coordinates of the extended vocabulary, only two entries can be non-zero; so: $q(z_t^i = x^i | x^i) = \alpha_t$ and $q(z_t^i = m | x^i) = 1 - \alpha_t$ and every other token has probability zero. At time t , the corrupted token is either the original token x^i or the mask token m .

This means that, at a fixed position i and diffusion time t , sampling z_t^i from its categorical marginal is equivalent to a **Bernoulli keep-or-mask** decision.

This translates to:

$$z_t^i \sim \begin{cases} x^i & \text{with probability } \alpha_t \\ m & \text{with probability } 1 - \alpha_t \end{cases}$$

We can write the full sequence distribution as:

$$q(\mathbf{z}_t | \mathbf{x}) = \prod_{i=1}^L q(z_t^i | x^i) = \prod_{i=1}^L \text{Cat}(z_t^i | \alpha_t \mathbf{e}(x^i) + (1 - \alpha_t) \mathbf{e}(m))$$

which means that positions are corrupted **independently** using the same retention probability α_t (making masking decisions **conditionally independent**).

While the marginal $q(\mathbf{z}_t | \mathbf{x})$ specifies the distribution of the corrupted sequence at each individual time t , to describe the complete forward process we need to specify how a state at an earlier time s transitions to a state at a later time t .

The mask token is an **absorbing state by construction**: if a token is already masked at time s , it remains masked at every later time t , with $0 \leq s < t \leq 1$:

$$q(z_t^i = m | z_s^i = m) = 1$$

where $q(z_t^i = m | z_s^i \neq m) = 1 - \frac{\alpha_t}{\alpha_s}$ is the conditional masking probability, while $q(z_t^i = z_s^i | z_s^i \neq m) = \frac{\alpha_t}{\alpha_s}$. $\frac{\alpha_t}{\alpha_s}$ is defined **conditional survival probability**.

The forward transition between times is defined as:

$$q(z_t^i | z_s^i) = \begin{cases} \text{Cat}(z_t^i | \mathbf{e}(m)) & z_s^i = m \\ \text{Cat}(z_t^i | \frac{\alpha_t}{\alpha_s} \mathbf{e}(z_s^i) + (1 - \frac{\alpha_t}{\alpha_s}) \mathbf{e}(m)) & z_s^i \neq m \end{cases}$$

Together with the absorbing transition $q(z_t^i = m \mid z_s^i = m) = 1$, this completely specifies the token-level forward transition from time s to time t . At sequence level:

$$q(\mathbf{z}_t \mid \mathbf{z}_s) = \prod_{i=1}^L q(z_t^i \mid z_s^i)$$

1.2 Reverse posterior

For $0 \leq s < t \leq 1$, having defined $q(z_s^i \mid x^i)$, $q(z_t^i \mid z_s^i)$ and $q(z_t^i \mid x^i)$, by Bayes' rule, we have that the exact reverse posterior is given by:

$$q(z_s^i \mid z_t^i, x^i) = \frac{q(z_t^i \mid z_s^i) \cdot q(z_s^i \mid x^i)}{q(z_t^i \mid x^i)}$$

This answers the question: given a clean token x^i and the token observed at the later noisier time t , what was the token at the earlier time s ?

The paper derives this $q(\mathbf{z}_s \mid \mathbf{z}_t, \mathbf{x})$ as:

$$q(z_s^i \mid z_t^i, x^i) = \begin{cases} \text{Cat}(z_s^i \mid \mathbf{e}(z_t^i)) & z_t^i \neq m \\ \text{Cat}\left(z_s^i \mid \frac{(1-\alpha_s)\mathbf{e}(m) + (\alpha_s - \alpha_t)\mathbf{e}(x^i)}{1 - \alpha_t}\right) & z_t^i = m \end{cases}$$

At sequence level, since the exact forward factorizes across positions:

$$q(\mathbf{z}_s \mid \mathbf{z}_t, \mathbf{x}) = \prod_{i=1}^L q(z_s^i \mid z_t^i, x^i)$$

We have two cases:

- if $z_t^i \neq m$, then the token has not yet been masked. Since the forward process neither changes one clean token into another one nor unmask a masked token, we must have $z_s^i = z_t^i = x^i$ with probability 1;
- if $z_t^i = m$, at an earlier time s it might have been masked ($P(z_s^i = m \mid x^i) = 1 - \alpha_s$) or it might still have been clean (and thus $P(z_s^i = x^i, z_t^i = m \mid x^i) = \alpha_s - \alpha_t$; in that case:

$$\begin{aligned} q(z_s^i = x^i \mid z_t^i = m, x^i) &= \frac{\alpha_s - \alpha_t}{1 - \alpha_t} \\ q(z_s^i = m \mid z_t^i = m, x^i) &= \frac{1 - \alpha_s}{1 - \alpha_t} \end{aligned}$$

1.3 Learned reverse transition

Since the true posterior depends on the clean sequence $\mathbf{x}$, which during generation is unknown (we only observe $\mathbf{z}_t$), we:

- introduce $\bar{x}_\theta^i(\mathbf{z}_t, t) = \begin{bmatrix} x_\theta^i(\mathbf{z}_t, t) \\ 0 \end{bmatrix}$, which predicts a probability distribution over the clean token x^i and mask as the final coordinate, so:

$$\bar{x}_\theta^i(\mathbf{z}_t, t) = p_\theta(x^i = \cdot \mid \mathbf{z}_t, t) \in \Delta^K$$

- consider the **learned reverse transition** obtained by substituting the model prediction for x^i .

The paper derives a closed form for this distribution, derived using the **SUBS parameterization**, which is:

$$p_\theta(z_s^i \mid \mathbf{z}_t) = \begin{cases} \text{Cat}(z_s^i \mid \mathbf{e}(z_t^i)) & z_t^i \neq m \\ \text{Cat}\left(z_s^i \mid \frac{(1-\alpha_s)\mathbf{e}(m) + (\alpha_s - \alpha_t)\bar{x}_\theta^i(\mathbf{z}_t, t)}{1 - \alpha_t}\right) & z_t^i = m \end{cases}$$

since the transformer predicts the clean token at position i using all of $\mathbf{z}_t$.

This parameterization requires two hard constraints (which we did not change):

1. **zero-masking probability**: the *clean* token **cannot** be $[MASK]$, meaning that $\langle \bar{x}_\theta^i(z_t, t), \mathbf{e}(m) \rangle = 0$; the $[MASK]$ logit is set to $-\infty$;

2. **carry-over unmasking**: when a position is already unmasked, the model output is replaced by a point mass on the observed token; this means that an already visible token is copied exactly. So, if $z_t^i \neq m$, we have $\langle \bar{x}_\theta^i(\mathbf{z}_t, t), \mathbf{e}(z_t^i) \rangle = 1$. This also makes it so the unmasked positions contributed zero to the loss, because:

$$-\log p_\theta(x^i = z_t^i | \mathbf{z}_t, t) = -\log 1 = 0$$

1.4 Training objective

Our goal is maximizing $\log p_\theta(x)$, which is the data log-likelihood. Since this likelihood requires marginalizing over the latent diffusion trajectory ($p_\theta(\mathbf{x}) = \sum_{\mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_T}} p_\theta(\mathbf{x}, \mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_T})$), it is generally intractable and thus we maximize the ELBO instead; in the paper, equivalently, in training we minimize the **negative ELBO** or, NELBO.

Also thanks to the SUBS parameterization, the authors of the paper show that the reconstruction term is identically zero and that the prior term is identically zero, making it so, if we define a discrete time grid $0 = t_0 < t_1 < \dots < t_T = 1$ and $s_j = t_{j-1}$

$$\mathcal{L}_{\text{NELBO}}^{(T)} = \sum_{j=1}^T \mathbb{E}_q \left[\frac{\alpha_{s_j} - \alpha_{t_j}}{1 - \alpha_{t_j}} \sum_{\ell=1}^L \text{CE}(x_\theta^\ell(\mathbf{z}_{t_j}, t_j), x^\ell) \right]$$

making it so training reduces to minimizing the weighted cross-entropy for predicting clean tokens at masked positions. Under carry-over unmasking, the unmasked positions contribute **zero**.

In the paper, they derive the **continuous-time weight** of the discrete coefficient $w_{s,t} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$; if we divide the time interval $[0, 1]$ into T intervals of width $\Delta t = \frac{1}{T}$, consider two consecutive times t and $s = t - \Delta t$ and apply the first-order Taylor expansion, we obtain:

$$\alpha_{t-\Delta t} = \alpha_t - \alpha'_t \Delta t + o(\Delta t)$$

Deriving that the discrete coefficient is approximately Δt times the continuous-time weight:

$$w(t) = \frac{-\alpha'_t}{1 - \alpha_t}$$

By defining $\alpha_t = e^{-\sigma(t)}$, we obtain:

$$w(t) = \frac{\sigma'(t)\alpha_t}{1 - \alpha_t} = \frac{\sigma'(t)}{e^{\sigma(t)} - 1}$$

This measures the rate at which the clean-token retention probability decreases, normalized by the current masking probability $1 - \alpha_t$.

Using the approximation above, we can write the NELBO as:

$$\mathcal{L}_{\text{NELBO}}^\infty = \mathbb{E}_{t \sim U(0,1), \mathbf{z}_t \sim q(\cdot | \mathbf{x})} [w(t) \text{CE}_t(\mathbf{x}, \mathbf{z}_t)]$$

1.5 Generation

The goal is implementing (almost directly) the reverse process described in the paper, considering a *conditional extension*.

Since we are testing generation on a question-answering setup, if we consider no conditioning, the model would sample an entire sequence from $p_\theta(x)$ starting from for example $[MASK], [MASK], \dots, [MASK]$ and it would not be forced to answer a specific user prompt (which is actually what we want).

Our task is then to model $p_\theta(x_{\text{answer}} | x_{\text{question}})$ instead: we treat the user question as **observed context** and apply diffusion only to the answer positions

To achieve this, we initialize, for $t = 1$ (so the first step in the *denoising* process) $x_1 = (x_{\text{question}}, [MASK], \dots, [MASK])$ while freezing the question throughout the reverse process.

If we define as $\mu_t = 1 - \alpha_t$ (the masking probability), if we have that the current token is masked, we only have two cases for $s = t - \Delta t$:

1. it remains masked with probability $\frac{\mu_s}{\mu_t} = P(x_s = m | x_t = m)$
2. we reveal the clean token with probability $\frac{\mu_t - \mu_s}{\mu_t}$

and, since we do not know the true clean token, we weight every possible token using $p_\theta(x_0|x_t)$ so that:

$$p_\theta(x_s = v|x_t) = \frac{\mu_t - \mu_s}{\mu_t} p_\theta(x_0 = v|x_t)$$

So the probability of x_s given x_t is given by:

$$p_\theta(x_s|x_t) = \underbrace{\frac{\mu_t - \mu_s}{\mu_t} p_\theta(x_0|x_t)}_{\text{reveal}} + \underbrace{\frac{\mu_s}{\mu_t} \delta_m}_{\text{stay masked}}$$

which answers the question: given the current noisy token, what should be the token one step earlier?

And while the neural network is responsible for deciding the identity of the token, the diffusion schedule decides whether is it time to reveal it, since:

- $\frac{\mu_t - \mu_s}{\mu_t} p_\theta(x_0|x_t)$ means "with probability $\frac{\mu_t - \mu_s}{\mu_t}$, reveal the token according to the transformer's predictions;
- $\frac{\mu_s}{\mu_t} \delta_m$ means "otherwise, keep the token masked"

and this depends on the **noise schedule** (since it depends on μ_t and μ_s). This is similar to the idea of *Inpainting using Denoising Diffusion Probabilistic Models* ("RePaint: Inpainting using Denoising Diffusion Probabilistic Models", Lugmayr et al., 2022)

One important thing to notice in this case is that the *confidence* the transformer has w.r.t. the different tokens is **ignored** when deciding *when* to reveal.

Given the assumptions of the SUBS parameterization, we can also eliminate *unnecessary* predictions: if a position is already visible, we already know its clean value.

So if $x_t^i \neq m$, then $x_0^i = x_t^i$ with certainty and therefore $p_\theta(x_0^i|x_t) = \delta_{x_t^i}$ (once a token is generated, it is never modified again).

So another nice thing is that, while autoregressive decoding computes $x_1 \rightarrow x_2 \rightarrow \dots$ one token after the other, at a certain diffusion step the transformer predicts $p_\theta(x_0^i|x_t)$ for every masked position **simultaneously** and then every position decides, *independently*, whether to remain masked or reveal its token.

2 Position-Dependent Noising

2.1 Vanilla MDLM

Let $x_0 = (x_1, \dots, x_L)$ be a tokenized sequence of length L .

In vanilla MDLM, each token is independently corrupted through a masking process. For every position i ,

$$q(z_t^i | x^i) = \text{Cat}(\alpha_t \mathbf{e}(x^i) + (1 - \alpha_t) \mathbf{e}(m))$$

where m denotes the mask token. The probability of preserving the original token is

$$P(z_t^i = x^i) = \alpha_t$$

while the probability of replacing it with a mask token is

$$P(z_t^i = m) = 1 - \alpha_t$$

The complete forward process factorizes across positions:

$$q(\mathbf{z}_t | \mathbf{x}) = \prod_{i=1}^L q(z_t^i | x^i)$$

Therefore, all positions share the same corruption schedule and are treated identically.

In the implementation,

$$1 - \alpha_t = 1 - e^{-\sigma(t)}$$

where $\sigma(t)$ is the continuous-time noise schedule.

2.2 Position-Dependent Corruption

Vanilla masked diffusion applies the same retention schedule α_t to every sequence position. Conditional on the clean sequence, every position therefore has the same probability of being masked at time t . Following the multivariate-noise framework of Garg et al., we instead assign a separate retention schedule to every position:

$$\alpha_{t,i}$$

The token-level forward marginal becomes

$$q(z_t^i | x^i) = \text{Cat}(z_t^i | \alpha_{t,i} \mathbf{e}(x^i) + (1 - \alpha_{t,i}) \mathbf{e}(m)).$$

Thus,

$$q(z_t^i = x^i | x^i) = \alpha_{t,i}$$

and

$$q(z_t^i = m | x^i) = 1 - \alpha_{t,i}$$

The sequence-level forward distribution still factorizes:

$$q(\mathbf{z}_t | \mathbf{x}) = \prod_{i=1}^L q(z_t^i | x^i)$$

Consequently, conditional on $\mathbf{x}$ and t , masking decisions remain independent across positions but are no longer identically distributed.

2.3 Position weights

We first define the normalized position

$$l_i = \frac{i - 1}{L - 1}$$

such that

$$\begin{aligned} l_1 &= 0 && \text{left-most token} \\ l_L &= 1 && \text{right-most token} \end{aligned}$$

We then introduce position-dependent weights defined as

$$\rho_i = 1 + \gamma \left(l_i - \frac{1}{2} \right)$$

where $0 \leq \gamma < 2$ controls the strength of the positional bias.

For example, if

$$\gamma = 0.2$$

then

$$\rho_1 = 0.9 \quad \rho_L = 1.1$$

Thus, positions on the left receive smaller weights while positions on the right receive larger weights (and so will be more likely to be masked).

2.4 Position-Dependent-Noise-Schedule

The corruption (noising) schedule is then defined as

$$\alpha_{t,i} = \alpha_t^{\rho_i} = e^{-\rho_i \sigma(t)}$$

so that the masking probability become

$$p_{\text{mask}}(t, i) = 1 - \alpha_t^{\rho_i}$$

Since $\rho_1 < 1$ and $\rho_L > 1$, we have that

$$p_{\text{mask}}(t, 1) < p_{\text{mask}}(t, L)$$

Therefore, rightmost tokens are more likely to be masked than leftmost tokens at the same diffusion time $t \rightarrow$ this introduces a right-to-left corruption bias.

The main idea is making it so the forward process consequently exhibits a right-to-left corruption bias and, equivalently, the reverse process is biased toward revealing left-hand positions before right-hand positions.

2.5 Why centering?

A simpler choice would be $\rho_i = 1 + \gamma \cdot l_i$; the main problem is that, in this case:

$$\rho_i \in [1, 1 + \gamma]$$

and the average weight becomes

$$\frac{1}{L} \sum_{i=1}^L \rho_i = 1 + \frac{\gamma}{2}$$

where $l_i = \frac{i-1}{L-1}$

As a consequence, the average masking probability would **increase** compared to vanilla MDLM and this would make the comparison unfair, since the modified model would receive a larger amount of noise overall.

To avoid this issue, we center the weights around 1:

$$\rho_i = 1 + \gamma \left(l_i - \frac{1}{2} \right)$$

So that the mean exponent is then exactly

$$\frac{1}{L} \sum_{i=1}^L \rho_i = 1$$

which keeps the average exponent equal to that of the vanilla schedule and prevents a systematic increase in the overall corruption strength (i.e. we do not simply *corrupt more*). This centering should be understood as an **approximate normalization**: because the mapping $\rho \mapsto \alpha_t^\rho$ is nonlinear, preserving the mean exponent does not preserve the mean masking probability exactly. Indeed,

$$\frac{1}{L} \sum_{i=1}^L \alpha_t^{\rho_i} \geq \alpha_t$$

and consequently

$$\frac{1}{L} \sum_{i=1}^L (1 - \alpha_t^{\rho_i}) \leq 1 - \alpha_t$$

Thus, the centered schedule generally produces slightly less masking on average than vanilla MDLM at intermediate times. For sufficiently small γ , centering provides a practical way to introduce a positional bias while limiting the accompanying change in the overall corruption level.

2.6 Training Objective

In the current implementation, the denoising architecture remains unchanged:

$$p_\theta(x^i \mid \mathbf{z}_t, t)$$

So now:

- the DiT architecture retains much of the architecture of the vanilla MDLM implementation;
- the SUBS parameterization is unchanged;
- the reverse diffusion process is retains the same functional form, but its coefficients are computed using the position-dependent schedule;

- the training loss retains the original scalar MDLM weighting:

$$\mathcal{L}_{\text{implemented}} = \mathbb{E}_{t, \mathbf{x}, \mathbf{z}_t} \left[w(t) \sum_{i=1}^L \mathbf{1}\{z_t^i = m\} \text{CE}(p_\theta(x^i | \mathbf{z}_t, t), x^i) \right],$$

where

$$w(t) = \frac{\sigma'(t)}{e^{\sigma(t)} - 1}.$$

Of in the code we enable the `position_loss_weighting`, we actually use **position-dependent weight** associated with the modified schedule.

- the forward corruption probabilities are replaced by the position-dependent probabilities

$$q(z_t^i = m | x^i) = 1 - \alpha_{t,i}$$

2.7 Interpretation

Vanilla MDLM uses the same marginal masking probability at every position:

$$q(z_t^i = m | x^i) = 1 - \alpha_t, \quad i = 1, \dots, L$$

Because the vanilla schedule is identical across positions, the forward process is permutation-invariant, while position-dependent schedules break this permutation symmetry, assigning greater probability to reveal orders in which leftmost positions are revealed earlier:

$$q(z_t^i = m | x^i) = 1 - \alpha_{t,i}$$

Because ρ_i increases from left to right and $0 < \alpha_t < 1$, rightmost positions have smaller retention probabilities and are therefore **more likely to be masked at a fixed time t** . The forward corruption process is thus **biased** toward masking rightmost positions earlier.

Consequently, the model more frequently observes examples of the form

$$(\text{visible left context}) \rightarrow (\text{masked right context})$$

which *encourages* a denoising strategy closer to autoregressive left-to-right prediction while remaining entirely within the diffusion framework.

3 Moving Sigmoid

The previous idea is:

$$p_{\text{mask}}(t, l) = 1 - \alpha_t^{\rho_i \sigma(t)}$$

where right positions have larger ρ_i (to implement the idea of masking right-to-left).

The **moving-sigmoid** construction implements the same right-to-left corruption bias through a different mechanism: a soft **masking front** that moves across the sequence. We introduce the preliminary masking schedule as:

$$p_{\text{mask}}(t, l) = \sigma(k(l_i - c(t)))$$

where

$$\sigma(u) = \frac{1}{1 + e^{-u}}$$

We have:

- l_i is the normalized position in $[0, 1]$
- t is the diffusion time in $[0, 1]$
- $k > 0$ controls *how sharp the transition is*
- $c(t)$ is the *location of the masking front*

The behaviour we look for is that, at early diffusion times, masking is concentrated near the right edge of the sequence. As t increases, the masking front moves leftward, making progressively more leftmost positions likely to be masked.

3.1 The uncalibrated sigmoid

We start from position normalization; we have that $l \in [0, 1]$ is:

$$l_i = \frac{i - 1}{L - 1}$$

where i is the token index and L the sequence length; we then have that $l_1 = 0$ is the leftmost token and $l = L = 1$ is the rightmost one.

A simple choice for the front location is $c_{\text{raw}}(t) = 1 - t$, which gives

$$c_{\text{raw}}(0) = 1 \text{ and } c_{\text{raw}}(1) = 0$$

so the front moves from the right edge toward the left edge as diffusion time increases. The corresponding raw masking probability is $\sigma(k(l_i + t - 1))$.

The parameter k determines the sharpness of the front: small values of k produce a smooth change in masking probability across positions, whereas large values produce a sharper transition resembling a moving boundary.

For example, with $L = 7$ and $k = 2$, the masking probabilities at $t = 0$ are approximately

$$[0.12 \quad 0.16 \quad 0.21 \quad 0.27 \quad 0.34 \quad 0.42 \quad 0.50]$$

With a larger value such as $k = 20$, the probabilities are close to zero at most positions, except near the right edge.

This uncalibrated construction exhibits our *desired spatial behavior*, but it has two important limitations:

1. the diffusion time t no longer determines the overall amount of corruption in the same way as in vanilla MDLM;
2. the raw sigmoid does not satisfy the standard endpoint conditions exactly. In particular,

$$p_{\text{mask}}^{\text{raw}}(0, L) = \frac{1}{2}$$

and

$$p_{\text{mask}}^{\text{raw}}(1, 1) = \frac{1}{2}$$

Consequently, at $t = 0$ the sequence is not almost surely clean, and at $t = 1$ it is not analytically guaranteed to be fully masked.

The uncalibrated schedule is therefore useful for illustrating the moving-front mechanism, but it does not provide a controlled replacement for the vanilla MDLM schedule.

3.2 A calibrated version

In Vanilla MDLM, we know that, at time step t , the average proportion of masked tokens is $\mu_t = 1 - \alpha_t$. So for $t = 0.1$ and a linear noise schedule, for example, we have roughly 10% of masked tokens.

The problem of the moving sigmoid is that we are not ensuring this: for $t = 1$ and for example $k = 10$, we have that

$$\bar{p}(t) = \int_0^1 \sigma(k(l + t - 1)) dl \rightarrow \bar{p}(1) \approx 0.93$$

We would need to find $c(t)$ such that:

$$\int_0^1 \sigma(k(l_i - c(t))) dl = 1 - \alpha_t$$

and, while there is a closed form (derived in the next section) we could solve this in practice considering:

$$p_{\text{mask}}(l_i) = \sigma(k(l_i - c))$$

so c controls where the boundary is; assuming a sufficiently large k , for example $c = 0.8$ would mean that only the far right gets masked, $c = 0.5$ that half the sequence gets masked and $c = 0.2$ means that most of the sequences gets masked.

If we want the average masking to be exactly $1 - \alpha_t$, we could:

- sample t , a diffusion time
- compute the target masking probability $\mu_t = 1 - \alpha_t$
- find the front position $c(t)$ that produces average masking μ_t
- use that $c(t)$ to sample the mask at every position;

Even though this binary search actually solves:

$$\frac{1}{L} \sum_{i=1}^L \sigma(k(l_i - c(t))) \approx \mu_t$$

So that then we approximately preserve *how much corruption there is* while also changing where it is concentrated.

3.3 Analytical calibrated version

We would need to find $c_{t,k}$ such that:

$$\int_0^1 \sigma(k(l_i - c_{t,k})) dl = 1 - \alpha_t$$

and we know, assuming $k \in \mathbb{R}, b \in \mathbb{R}$, that:

$$\int \sigma(k_i l + b) dl = \frac{\ln(1 + e^{k_i l + b})}{k} + C$$

The, to find the correct $c_{t,k}$ we write the explicit integration and we solve for $c_{t,k}$.

$$\begin{aligned} \frac{\ln(1 + e^{k(1-c_{t,k})})}{k} - \frac{\ln(1 + e^{-kc_{t,k}})}{k} &= 1 - \alpha_t \\ \ln\left(\frac{1 + e^{k(1-c_{t,k})}}{1 + e^{-kc_{t,k}}}\right) &= k(1 - \alpha_t) \\ \frac{1 + e^{k(1-c_{t,k})}}{1 + e^{-kc_{t,k}}} &= e^{k(1-\alpha_t)} \\ 1 + e^k e^{-kc_{t,k}} &= e^{k(1-\alpha_t)} (1 + e^{-kc_{t,k}}) \\ e^k e^{-kc_{t,k}} - e^{k(1-\alpha_t)} e^{-kc_{t,k}} &= e^{k(1-\alpha_t)} - 1 \\ (e^k - e^{k(1-\alpha_t)}) e^{-kc_{t,k}} &= e^{k(1-\alpha_t)} - 1 \\ e^{-kc_{t,k}} &= \frac{e^{k(1-\alpha_t)} - 1}{e^k - e^{k(1-\alpha_t)}} \\ c_{t,k} &= \frac{1}{k} \ln \frac{e^k - e^{k(1-\alpha_t)}}{e^{k(1-\alpha_t)} - 1} \end{aligned}$$

Then we compute the loss weight. Base on the definition of the MLDM paper:

$$\text{Weight}_{t,i} = \frac{\frac{\partial}{\partial t} p_{\text{mask}}(t, i)}{p_{\text{mask}}(t, i)}$$

Let's apply this to the moving sigmoid function:

$$p_{\text{mask}}(t, i) = \sigma(k(l_i - c_t))$$

$$\frac{\partial p_{\text{mask}}}{\partial t} = p_{\text{mask}}(1 - p_{\text{mask}}) \cdot (-k \cdot c'_t)$$

Back into the formula:

$$\text{Weight}_{t,i} = \frac{p_{\text{mask}}(1 - p_{\text{mask}})(-k \cdot c'_t)}{p_{\text{mask}}} = -k(1 - p_{\text{mask}})c'_t$$

with:

$$c'_t = -m'_t e^{km_t} \frac{e^k - 1}{(e^k - e^{km_t})(e^{km_t} - 1)}$$

4 Introduction to the Evaluation Paradigm

The evaluation of generative language models has evolved significantly with the advent of novel architectures. While Autoregressive (AR) models factorize the joint probability of a sequence directionally, Discrete Diffusion Models (DDMs) destroy and reconstruct sequences through stochastic markov transitions. This structural divergence mandates a dual-pronged benchmarking framework. This report provides a mathematically rigorous exploration of the evaluation methods implemented in our benchmarking suite, dissecting both the generative trajectory and the post-generation semantic fidelity.

4.1 Part I: Intrinsic Dynamics During Generation

Unlike AR models, where generation is an immutable left-to-right append operation, DDMs iteratively resolve a sequence of tokens of fixed length. Monitoring the internal state during this T -step denoising process offers critical insights into the convergence behavior of the unmasking schedule.

4.2 Trajectory Analysis of Discrete Diffusion

Let a sequence of discrete tokens be $X = (x_1, x_2, \dots, x_N)$. In a masked DDM, the forward noising process $q(X_t|X_{t-1})$ gradually replaces true tokens with a special [MASK] token according to a schedule α_t . The reverse generation process $p_\theta(X_{t-1}|X_t)$ attempts to predict the true categorical distribution of the masked tokens.

4.2.1 Masked Shannon Entropy

To quantify the model's epistemic uncertainty during generation, we compute the Shannon Entropy strictly over the subset of tokens that remain masked at step t . Let M_t be the set of indices where $x_{i,t} = [\text{MASK}]$. For each masked position, the model outputs a categorical probability distribution $p_\theta(x|X_t)$ over the vocabulary V .

$$H(X_t) = -\frac{1}{|M_t|} \sum_{i \in M_t} \sum_{v \in V} p_\theta(x_i = v | X_t) \log p_\theta(x_i = v | X_t)$$

Motivation: Entropy serves as a proxy for schedule efficacy. A well-calibrated unmasking policy should exhibit a monotonic decrease in entropy. If entropy collapses too early, the model is overconfident and prone to structural hallucinations. If it remains high until $t \rightarrow 0$, the schedule is too conservative.

4.2.2 Step-Perplexity (Pseudo-Perplexity) Dynamics

While Entropy measures internal confidence, it does not guarantee correctness. The model could be highly confident but entirely wrong. To measure the instantaneous trajectory of accuracy, we introduce **Step-Perplexity** (Pseudo-Perplexity). At each step t , we evaluate the cross-entropy loss exclusively on the currently masked tokens M_t against the known ground-truth labels Y .

$$PP(X_t) = \exp \left(-\frac{1}{|M_t|} \sum_{i \in M_t} \log p_\theta(x_i = y_i | X_t) \right)$$

Motivation and Algorithmic Logic: In the `_calculate_step_ppl` method, we isolate `logits[masked_indices]` and `target_labels[masked_indices]`. This is crucial because including already unmasked (resolved) tokens would artificially deflate the perplexity, giving a false sense of performance. Tracking $PP(X_t)$ allows us to visualize exactly when the model "figures out" the underlying structure of the target text during the diffusion process.

4.2.3 Semantic Convergence (Self-Similarity)

We analyze how the hidden manifold representations converge toward the final grounded meaning. Let $h_t \in \mathbb{R}^{N \times d}$ be the last hidden state of the transformer at step t . We apply average pooling across the sequence length N to obtain a global sentence embedding $z_t \in \mathbb{R}^d$.

$$\text{CosSim}(z_t, z_0) = \frac{z_t \cdot z_0}{\|z_t\|_2 \|z_0\|_2}$$

IMPLEMENTATION CHALLENGE: GPU Memory Saturation

Tracking high-dimensional states h_t across T steps for an entire batch requires $O(T \times B \times N \times d)$ memory, which rapidly exceeds VRAM limits (OOM). The algorithmic solution implemented enforces aggressive detachment and dimensionality reduction on the fly: `pooled_state = last_hidden_state.mean(dim=1).detach().cpu()`. This restricts the memory footprint to $O(T \times B \times d)$ in system RAM, discarding the autograd graph.

5 Part II: Extrinsic Post-Generation Evaluation

Once the sequences are fully decoded, we evaluate the holistic probability distributions and the semantic validity of the conversational outputs against human references.

5.1 Perplexity and The Evidence Lower Bound (ELBO)

2.1 Exact Likelihood (Autoregressive)

AR models define the joint probability using the chain rule of probability. The exact negative log-likelihood (NLL) for a sequence of length N is computationally tractable:

$$NLL_{AR}(X) = -\sum_{i=1}^N \log p_{\theta}(x_i | x_{<i})$$

Perplexity is strictly defined as $PPL = \exp\left(\frac{1}{N} \cdot NLL_{AR}(X)\right)$.

5.1.1 Variational Bounds (Discrete Diffusion)

For a DDM, computing exact likelihood requires marginalizing over all possible unmasking trajectories ($O(N!)$ operation). Therefore, we optimize and evaluate using the Evidence Lower Bound (ELBO). For a continuous timestep $t \sim U(0, 1)$, the simplified continuous-time ELBO is derived as the weighted expectation of the cross-entropy loss over masked tokens:

$$\mathcal{L}_{ELBO} = \mathbb{E}_t[w(t) \cdot CE(X_{true}, p_{\theta}(X | X_{mask}(t)))]$$

5.2 Deep Semantic Evaluation (Micro and Macro)

Surface-level lexical metrics (like standard BLEU or ROUGE) are notoriously poor at capturing paraphrased meaning. Our framework implements a two-tiered semantic evaluation: Micro (Instance-level) via BERTScore, and Macro (Distributional) via Fréchet BERT Distance.

5.2.1 Micro-Semantic Matching: BERTScore

BERTScore computes token-level similarity between the generated hypothesis $\hat{x}$ and the human reference x using deep contextual embeddings from a pre-trained transformer model. Rather than requiring exact string matches, it calculates the cosine similarity between the contextualized vector of each token in the hypothesis against every token in the reference.

The algorithm employs a greedy bipartite matching to find the maximum alignment. Recall (R_{BERT}) and Precision (P_{BERT}) are calculated as:

$$\begin{aligned} R_{BERT} &= \frac{1}{|x|} \sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} x_i^T \hat{x}_j \\ P_{BERT} &= \frac{1}{|\hat{x}|} \sum_{\hat{x}_j \in \hat{x}} \max_{x_i \in x} x_i^T \hat{x}_j \end{aligned}$$

Motivation: BERTScore is indispensable because it assigns high scores to semantically equivalent but lexically divergent responses (e.g., "The weather is freezing" vs "It is extremely cold outside"). We combine Precision and Recall to extract the $F1$ score as our primary micro-semantic metric.

5.2.2 Macro-Distributional Topology: Fréchet BERT Distance (FBD)

While BERTScore evaluates isolated pairs, FBD measures the topological distance between the entire real and generated semantic manifolds. We treat the embeddings of the conversations as samples from multivariate Gaussian distributions $X_r \sim \mathcal{N}(\mu_r, \Sigma_r)$ and $X_g \sim \mathcal{N}(\mu_g, \Sigma_g)$.

$$FBD = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Motivation for Joint Encodings: To capture conversational coherence, the input to the BERT encoder is explicitly concatenated as "Prompt [SEP] Response". This ensures the covariance matrices Σ capture the conditional dependency between the user's question and the model's answer, directly penalizing non-sequiturs.

IMPLEMENTATION CHALLENGE: Numerical Instability of Matrix Roots

The term $(\Sigma_r \Sigma_g)^{1/2}$ requires computing the principal square root of a matrix via `scipy.linalg.sqrtm`. Because empirical covariance matrices computed from high-dimensional embeddings (e.g., 768 dimensions for BERT-base) are often ill-conditioned or rank-deficient, floating-point truncation introduces imaginary components. The algorithm strictly requires an imaginary safeguard: `if np.iscomplexobj(covmean): covmean = covmean.real` to project the result back to the real domain, avoiding cascading NaNs.

5.3 Lexical and Structural Diversity Evaluation

Generative models are highly susceptible to "mode collapse." They learn safe, generic templates and repeat them unconditionally.

5.3.1 N-Gram Type-Token Ratio (TTR)

To measure vocabulary saturation, we compute the Unique N -gram Ratio:

$$\text{Unique Ratio} = |\text{Set}(G_n)| / \overline{|G_n| \times 100}$$

4.2 Self-BLEU Repetitiveness

Self-BLEU treats the model as its own adversary. For a corpus of S generated sentences, the score for sentence s_i is calculated by treating the remaining $S - 1$ sentences as the ground-truth reference set.

IMPLEMENTATION CHALLENGE: Asymptotic Complexity & Smoothing

Computing exact Self-BLEU requires $O(|S|^2)$ pairwise BLEU calculations. The algorithm utilizes random subsampling without replacement (`torch.random.sample`) capped at 1000. Furthermore, standard NLTK BLEU enforces 4-gram overlap. Short conversational responses (e.g., "I don't know.") contain zero 4-grams, causing a zero-division crash. The strict inclusion of `SmoothingFunction().method1` mathematically prevents this collapse.

5.4 The LLM-as-a-Judge Paradigm

Traditional metrics fail to reliably correlate with human preference for complex reasoning. We employ an instruction-tuned LLM as an adjudicator utilizing a zero-shot Bradley-Terry formulation.

5.4.1 The Bradley-Terry Preference Model

The probability that a judge prefers response A over B is given by $P(A \succ B) = \frac{\exp(s_A)}{\exp(s_A) + \exp(s_B)}$, where s is the latent quality score.

IMPLEMENTATION CHALLENGE: Position Bias and Parsing

LLM judges suffer from extreme "Position Bias"—an algorithmic predisposition to favor the first sequence presented. If Model A is consistently first, statistics will skew. The orchestrator explicitly neutralizes this by implementing a randomized swap (`swap = random.choice([True, False])`) prior to prompt construction, followed by a post-generation de-swap.

References

- [1] S. Sahoo *et al.*, “Simple and effective masked diffusion language models,” *ICLR*, 2024.